Accuracy–Hardware Complexity Trade-offs in Dynamic Branch Predictors: A Comparative Analysis from Saturating Counters to Tournament Predictors

Aman Sharma (2024EE30115)
Atharv Khedkar (2024EE30289)
Makar Vishwajit Shankar (2024MT11174)
Om Sahu (2024MT10764)
Sai Kaushik Tati (2024MT60849)

Computer Organization and Architecture (Term Paper)

Abstract

A wrong branch guess in a pipelined processor forces the pipeline to discard speculatively fetched instructions, so the accuracy of the branch predictor directly limits how much of the pipeline's potential can be used. This paper is a literature survey of the classical lineage of dynamic branch predictors (1981–1999): the saturating-counter schemes of Smith (1981), the two-level adaptive predictors of Yeh and Patt (1991, 1993), the gshare predictor and the principle of combining predictors of McFarling (1993), and the tournament predictor of the Alpha 21264 (Kessler, 1999). We compare these designs along two axes: the accuracy each is reported to achieve and the hardware cost it requires, measured as bits of predictor state (table entries $\times$ counter or history width). We derive closed-form state-cost expressions for each family, evaluate them on the one fully specified real design (the Alpha 21264, about 29 Kbit of table state), and analyse why each step in the lineage was introduced. We do not run new experiments; we report only quantitative accuracy claims that we could trace to a primary source, and we state clearly where the available sources do not support a like-for-like numerical comparison.

Copyright Assignment

All intellectual property rights (IPR) in this report, including but not limited to the text, original analysis, tables, and figures produced by the authors, are assigned to the Indian Institute of Technology Delhi (IIT Delhi).

Where material from external sources has been cited, reproduced, or adapted in this report (including but not limited to research papers, textbooks, and other published literature listed in the References section), the intellectual property rights to that material remain with the respective original authors/owners, and such material has been used here for academic and non-commercial purposes only, with due attribution.

Acknowledgements

We would like to thank Prof. Sumantra DuttaRoy for guidance on scoping this term paper. We also acknowledge the Department of Electrical Engineering and the Indian Institute of Technology Delhi for the academic environment, library access, and computing resources that supported this work.

Contents

1 Introduction **4**
1.1 Research questions . . . 4
1.2 Scope . . . 4
1.3 Method and contribution . . . 4

2 Background: Why Prediction Accuracy Matters **4**
2.1 Static versus dynamic prediction . . . 5

3 The Comparison Metric **5**

4 The Four Designs **5**
4.1 Saturating counters (Smith, 1981) . . . 5
4.2 Two-level adaptive prediction (Yeh and Patt, 1991, 1993) . . . 6
4.3 Gshare and combining predictors (McFarling, 1993) . . . 6
4.4 The Alpha 21264 tournament predictor (Kessler, 1999) . . . 6

5 A Unified State-Cost Model **7**
5.1 Worked example: the Alpha 21264 . . . 7

6 Analysis: The Logic of the Evolution **8**
6.1 Answering the research questions . . . 8

7 Discussion and Limitations **9**

8 Conclusion and Future Work **10**

A Plagiarism Report **10**

B Reading Indices **10**

List of Figures

1 Structure of the Alpha 21264 tournament predictor, with table sizes as described by Kessler (1999) (local counter width from Hennessy and Patterson). The chooser steers the multiplexer between the local and global predictions. . . . 7
2 Predictor state versus history length for a single 2-bit-counter PHT. State doubles with each additional history bit. This is a purely analytical curve; it says nothing about accuracy. . . . 9

List of Tables

1 State-cost expressions for each predictor family (derived by the authors from the structural descriptions in the cited papers). . . . 8
2 State of the Alpha 21264 tournament predictor (≈29 Kbit). The 12-bit global history register adds 12 further bits. Local counter width of 3 bits is from Hennessy and Patterson. . . . 8
3 The chain of design motivations along the lineage. Entries paraphrase the motivations stated in the cited papers. . . . 9

1 Introduction

A pipelined processor fetches a new instruction every cycle, long before the previous instructions have finished executing. A conditional branch breaks this flow: the processor does not know which instruction to fetch next until the branch condition is evaluated, several stages later. This is a *control hazard* (Sarangi, Ch. 10, §10.4.3). Waiting for every branch to resolve would waste cycles on a large fraction of the instruction stream, so processors instead *predict* the branch direction and keep fetching speculatively. When the guess is wrong, the speculatively fetched instructions are discarded in a *pipeline flush*, and the cost of that flush grows with pipeline depth.

The prescribed textbook introduces branch prediction as the principal remedy (Sarangi, §10.11.1), covering static prediction, saturating counters, and correlating predictors at an introductory level. It does not explain why the field moved from simple counters to history-based and hybrid schemes, nor what each step bought in accuracy relative to its hardware cost. This paper addresses that gap.

1.1 Research questions

We ask one overarching question: *how did dynamic branch predictor design evolve between 1981 and 1999, and what did each step buy in accuracy relative to the hardware it cost?* It is broken into four sub-questions:

RQ1. What limitation of single saturating counters motivated the move to history-based (two-level) predictors?

RQ2. Why does indexing with global history, and specifically hashing it with the branch address (gshare), help?

RQ3. What design principle underlies combining predictors, and what does it cost?

RQ4. How does accuracy scale with hardware budget, and what does the Alpha 21264 tell us about a real implementation?

1.2 Scope

The paper follows four related design ideas: (i) the saturating-counter scheme of Smith (1981); (ii) the two-level adaptive predictor of Yeh and Patt (1991) and its systematic comparison by Yeh and Patt (1993); (iii) gshare and the principle of combining predictors, due to McFarling (1993); (iv) the tournament predictor of the DEC/Compaq Alpha 21264 (Kessler, 1999). The scope is deliberately bounded to this classical lineage. Perceptron-based and TAGE predictors are excluded and are mentioned only as future directions in Section 8. This is a survey and written analysis; it contains no new simulation.

1.3 Method and contribution

We read the primary sources, extract the structural description of each predictor, and define a common metric (Section 3). The analytical contribution is the unified state-cost model of Section 5, which makes the designs comparable on one axis, together with the chain of design motivations in Section 6.

2 Background: Why Prediction Accuracy Matters

Let f_b be the fraction of executed instructions that are conditional branches, m the misprediction rate (mispredictions per branch), and P the misprediction penalty in cycles. If the pipeline would

otherwise achieve a base cycles-per-instruction of CPI_0, then to first order

$$\mathrm{CPI} = \mathrm{CPI}_0 + f_b \cdot m \cdot P. \tag{1}$$

The model is simple but exposes the two trends that drove the field. A deeper pipeline raises P, and a wider machine lowers CPI_0, so the branch term becomes a larger share of total time. As a purely hypothetical illustration (these numbers are not measurements), take $f_b = 0.2$, $P = 10$, and $\mathrm{CPI}_0 = 0.5$. Reducing m from 0.10 to 0.05 lowers the branch term from 0.20 to 0.10 cycles per instruction, i.e. it reduces CPI from 0.70 to 0.60, about a 14% improvement. Every fraction of a percent of accuracy therefore has a measurable performance value, which is why designers accept substantial hardware for the predictor.

The other side of the trade-off is that predictor tables occupy chip area, consume power, and must be read within the fetch stage, so larger is not free. The comparison in this paper is a comparison of how effectively each design converts storage into accuracy.

2.1 Static versus dynamic prediction

Static prediction fixes the guess at compile time or by a simple rule (for example, "backward branches are taken"). *Dynamic* prediction uses hardware that observes past outcomes at run time and adapts. All four designs studied here are dynamic.

3 The Comparison Metric

We define *hardware complexity* as the number of bits of predictor state: table entries multiplied by counter or history width, summed over every table in the design. We chose this because it is reportable for all five source papers, independent of circuit technology, and directly comparable across designs.

What the metric leaves out. Bits of state do not capture access latency (a large table is slower to read), the number of ports, the logic that computes indices and updates counters, or tags and checkpointing for speculative history update. Two designs with equal state may differ in timing difficulty: for example, a design needing two dependent table lookups per prediction is harder to fit in one cycle than one needing a single lookup. We therefore treat bits of state as a first-order cost and flag the second-order costs where the sources discuss them.

4 The Four Designs

4.1 Saturating counters (Smith, 1981)

Smith (1981) analysed several branch prediction strategies using instruction-trace data and proposed, among others, keeping a small table of counters indexed by (a hash of) the branch address. A *saturating counter* of n bits counts up on a taken branch and down on a not-taken branch, but stops at its minimum and maximum values. The predicted direction is taken if the counter is in its upper half. With $n = 2$ there are four states (strongly not taken, weakly not taken, weakly taken, strongly taken), so a single anomalous outcome (such as the final iteration of a loop) does not immediately reverse a well-established prediction. This hysteresis is the key advantage over a 1-bit "last outcome" scheme, which mispredicts twice per loop execution.

Limitation. A counter records a branch's *bias*, but not its *pattern*. A branch that alternates taken, not taken, taken, not taken is perfectly predictable if one knows the previous outcome, yet a 2-bit counter hovers near the middle and is wrong often. Distinct branches that hash to the same entry (*aliasing*) also corrupt one another's counters. These limitations motivate history-based prediction (RQ1).

4.2 Two-level adaptive prediction (Yeh and Patt, 1991, 1993)

Yeh and Patt (1991) introduced the *two-level adaptive* predictor. The first level records the recent outcomes of branches in one or more *branch history registers* (BHRs). The second level is a *pattern history table* (PHT) of small counters indexed by the history pattern. The predictor therefore learns, for each pattern of recent outcomes, what the branch usually does next. For the alternating branch above, the history "taken" maps to a counter that learns "not taken" and vice versa, giving near-perfect accuracy after warm-up.

Yeh and Patt (1993) compared a family of variants systematically. The variants differ in two choices: whether the first-level history is *global* (one register shared by all branches) or *per-address* (one register per branch, or per set of branches), and whether the PHT is shared or per-address. In the nomenclature of that work these appear as names such as GAg, PAg, and PAp, where the first letter gives the history type and the last letter gives the PHT organisation (the exact definitions should be checked against the original paper before the final submission, as only the abstract-level structure of this paper has been examined by the authors).

Cost behaviour. The PHT has 2^h entries for an h-bit history, so state grows exponentially in history length; long histories capture longer patterns but cost much more, and fill slowly (each pattern must be learned separately). This tension drives the cost analysis in Section 5.

4.3 Gshare and combining predictors (McFarling, 1993)

McFarling (1993) reviews bimodal, local, and global predictors, then makes two contributions relevant here.

Gshare. A global-history predictor indexed by history alone cannot distinguish different branches that happen to see the same recent history, while concatenating address and history bits ("gselect") wastes index bits when the two are partly redundant. Gshare instead XORs the global history with the branch address bits to form the index into a single table of 2-bit counters. The motivating observation is that few distinct paths usually lead to a given branch, so the address and the history are highly correlated; the XOR then spreads accesses over the whole table without dedicating separate index bits to each. For a table of the same size, this reduces destructive aliasing relative to history-only indexing. Gshare also needs only one table lookup, which makes it fast.

Combining predictors. No single predictor is best for all branches: some branches are strongly biased and need only a bimodal counter, while others depend on history and benefit from gshare or a local predictor. McFarling's technique runs two predictors in parallel and adds a third table, a *chooser* of 2-bit counters, which records for each branch which component has been more accurate and selects its prediction. The chooser is updated only when the two components disagree, so that it moves toward whichever was correct. The state cost is the sum of the component tables plus the chooser, a premium paid for adaptivity across branch types (RQ3).

4.4 The Alpha 21264 tournament predictor (Kessler, 1999)

The Alpha 21264 implements a *tournament* predictor, which Kessler (1999) describe as choosing dynamically between a local-history and a global-history predictor, and which they report to be more accurate than larger tables of either individual method, with a 90–100% success rate on most simulated benchmarks. The structure (Figure 1) is:

- **Local predictor (two-level).** A first-level table holds 10 bits of history for each of 1,024 branches. The 10-bit pattern selects one of 1,024 prediction counters. The counters are 3 bits

wide in the treatment of Hennessy and Patterson, Ch. 3; the primary-source text we consulted specifies the number of counters but not their width, so the 3-bit figure is taken from this secondary source.

- **Global predictor.** A 4,096-entry table of 2-bit saturating counters, indexed by the global (path) history of the last 12 branches.
- **Chooser.** A 4,096-entry table of 2-bit counters, also indexed by the path history, selecting between the local and global predictions.

Kessler (1999) also describe the design rationale: some branches correlate with their own earlier behaviour (local history), others with the behaviour of earlier branches (global history), and both kinds of correlation are needed for maximum accuracy. The same predictor is described in the longer microarchitecture paper of Kessler et al. (1998).

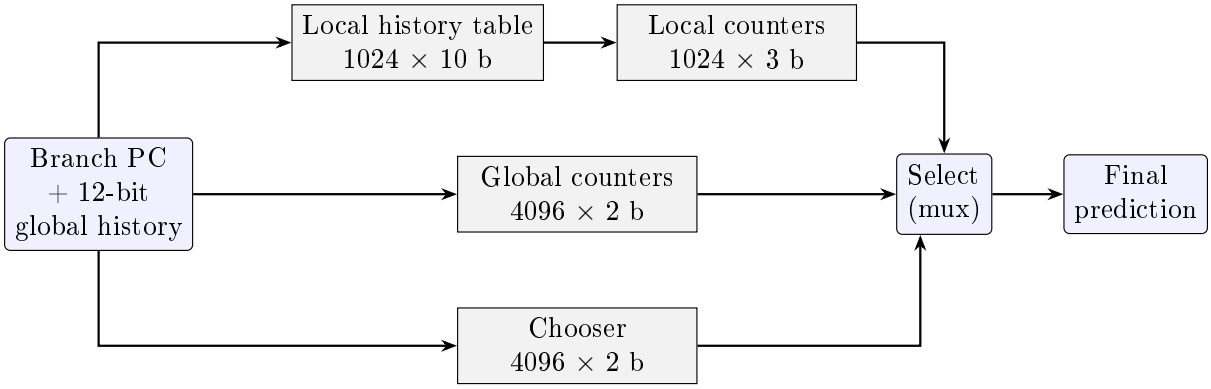


Figure 1: Structure of the Alpha 21264 tournament predictor, with table sizes as described by Kessler (1999) (local counter width from Hennessy and Patterson). The chooser steers the multiplexer between the local and global predictions.

5 A Unified State-Cost Model

Using the metric of Section 3, we derive the state in bits for each family. These formulas are the authors' own analysis of the designs as described in the sources; they are not figures quoted from them. All counters are assumed 2-bit unless noted, and tags and speculative-update checkpoints are ignored.

Reading the formulas. The terms make the design trade-offs explicit. In the global two-level and gshare designs the PHT term $2 \cdot 2^k$ dominates, doubling for every extra history bit, whereas in the per-address designs the register term Bh grows only linearly in B and h. The PAp-style design multiplies the exponential PHT term by B, which is why shared-PHT designs are attractive when storage is tight. Figure 2 plots the exponential growth for the global-history family.

5.1 Worked example: the Alpha 21264

Applying the metric to the sizes reported in Section 4.4: The local predictor (history table plus counters) accounts for 13,312 of 29,696 bits, about 45%. The chooser alone costs as much as the entire global predictor, so adaptivity between the two predictors costs roughly as much state as one of the predictors it chooses between. According to Kessler (1999), this arrangement outperforms larger tables of either individual method, which is the central empirical justification for the extra chooser state.

Design	Parameters	State (bits)
Bimodal (Smith)	N counters of n bits	$S_{\text{bim}} = nN$
Global two-level (GAg-style)	k-bit global history; PHT of 2^k 2-bit counters	$S_{\text{GAg}} = k + 2 \cdot 2^k$
Per-address history, shared PHT (PAg-style)	B history registers of h bits; one PHT of 2^h counters	$S_{\text{PAg}} = Bh + 2 \cdot 2^h$
Per-address history, per-address PHT (PAp-style)	B registers of h bits; B PHTs of 2^h counters	$S_{\text{PAp}} = Bh + 2B \cdot 2^h$
Gshare (McFarling)	m-bit history XOR address; 2^m counters	$S_{\text{gs}} = m + 2 \cdot 2^m$
Combined bimodal + gshare (McFarling)	N_b bimodal, 2^m gshare, N_c chooser entries	$S_{\text{comb}} = 2N_b + m + 2 \cdot 2^m + 2N_c$
Alpha 21264 tournament (Kessler)	as in Section 4.4	see Table 2

Table 1: State-cost expressions for each predictor family (derived by the authors from the structural descriptions in the cited papers).

Component	Entries × width	Bits	Share
Local history table	1024 × 10	10,240	34.5%
Local counters	1024 × 3	3,072	10.3%
Global counters	4096 × 2	8,192	27.6%
Chooser	4096 × 2	8,192	27.6%
Total (tables)		**29,696**	100%

Table 2: State of the Alpha 21264 tournament predictor (≈29 Kbit). The 12-bit global history register adds 12 further bits. Local counter width of 3 bits is from Hennessy and Patterson.

6 Analysis: The Logic of the Evolution

Each step in the lineage can be read as a response to a specific failure of the previous design (Table 3).

6.1 Answering the research questions

RQ1. Single counters track bias and cannot represent patterns such as alternation or fixed-length loop behaviour; adding a history level lets the predictor map patterns to outcomes (Yeh and Patt, 1991).

RQ2. Global history exposes correlation between different branches, but indexing by history alone or by concatenation either aliases branches or wastes index bits. XORing history with the address uses all index bits and exploits the fact that address and history are often redundant (McFarling, 1993).

RQ3. Different branches favour different predictors. Combining two predictors with a small

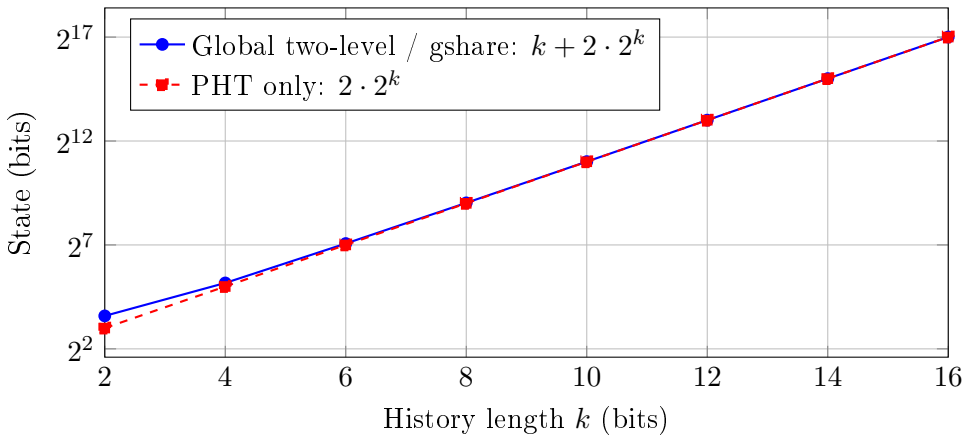


Figure 2: Predictor state versus history length for a single 2-bit-counter PHT. State doubles with each additional history bit. This is a purely analytical curve; it says nothing about accuracy.

Step	**Problem addressed**	**Mechanism**	**Cost incurred**
Smith 1981 (2-bit counter)	1-bit scheme double-mispredicts at loop exits	Hysteresis from a 2-bit saturating counter	2 bits per entry; no pattern knowledge
Yeh & Patt 1991 (two-level)	Counters capture bias, not patterns	History register(s) select a pattern-specific counter	Exponential PHT growth in history length
McFarling 1993 (gshare)	History-only indexing aliases; concatenation wastes index bits	XOR address with global history	Same table size; one extra XOR
McFarling 1993 (combining)	No single predictor suits all branches	Chooser selects between two components	Two components plus a chooser table
Kessler 1999 (21264)	Both local and global correlation are needed	Tournament of a local and a global predictor	≈29 Kbit total

Table 3: The chain of design motivations along the lineage. Entries paraphrase the motivations stated in the cited papers.

chooser lets each branch use the more suitable one, at a cost of one extra table (McFarling, 1993).

RQ4. For equal total state, the cost model shows exponential growth in PHT size with history length, so extra bits are best spent where they buy the most; the 21264 spends roughly half of its state on local history and half on global history plus chooser (Kessler, 1999). Kessler's claim that the tournament scheme beats larger tables of either single method is the strongest evidence in the sources for the view that structure matters more than raw size.

7 Discussion and Limitations

Accuracy comparison across papers. The papers use different benchmarks, trace lengths, and warm-up conventions, and the 1993 Yeh–Patt comparison and McFarling's note do not report on a common benchmark suite. We therefore do *not* construct a single accuracy-versus-bits table from numbers pulled from different papers, because such a table would imply a like-for-like comparison that the sources do not support. The only accuracy figure we quote is Kessler's

90–100% range for most simulated benchmarks, together with the qualitative result that the tournament predictor beats larger single-method tables. A quantitative accuracy axis would require rerunning all designs on one trace set, which is outside the scope of this survey (this was the optional simulation work package in the project plan, which we chose not to pursue to keep the paper a literature survey).

Metric limitations. As noted in Section 3, bits of state ignore timing and logic cost. For example, gshare's single lookup is faster than the two-level local predictor's two dependent lookups, which matters for fetch-stage prediction but is invisible to our metric.

Source coverage. Two of the five primary sources (Smith, 1981; McFarling, 1993) were read in full by the authors; the other three were studied mainly at the level of structure and abstract, with the 21264 description cross-checked against two versions of Kessler's papers. Details of the Yeh–Patt nomenclature in particular should be verified against the originals.

8 Conclusion and Future Work

The classical predictors from 1981 to 1999 form a coherent progression in which every step answers a concrete weakness of its predecessor: hysteresis from 2-bit counters, pattern learning from two-level history, better index use from gshare, and per-branch adaptivity from combining predictors, culminating in the 29 Kbit Alpha 21264 tournament predictor. Measured in bits of state, the designs span from 2 bits per entry to tens of kilobits, with exponentially growing tables as the main cost driver, and the tournament design shows that spending a portion of the budget on a chooser can be more effective than enlarging a single predictor.

Future work. Later predictors, such as perceptron-based predictors and TAGE, extend the same themes (longer histories at manageable cost, adaptive selection among components) and would form a natural continuation. A common-trace simulation of the four designs would supply the missing accuracy axis.

A Plagiarism Report

This report is to be checked with Grammarly's Plagiarism Checker (free tier, in passes under the per-scan word limit) and, once access is available through the IIT Delhi library, with Turnitin. The results of the check on this final text must be inserted here before submission. The analytical framing (the bits-of-state comparison axis, the four-idea lineage, the unified cost model, and the research questions) is the authors' own. Technical facts about each predictor are drawn from the cited papers (Smith, 1981; Yeh and Patt, 1991; McFarling, 1993; Yeh and Patt, 1993; Kessler, 1999) and the cited textbooks, and are attributed accordingly; all descriptions are written in the authors' own words.

B Reading Indices

The readability indices for the final prose should be recomputed on this final text (headings, tables, captions, and bibliography excluded) with the same syllable-counting script used for Draft 1, and the table inserted here.

Glossary

branch prediction A hardware technique for guessing the outcome of a conditional branch before it is resolved, so the pipeline can keep fetching instructions speculatively.

control hazard A pipeline hazard caused by branch and jump instructions, where the next instruction to fetch is not known until the branch is resolved.

gshare A predictor that indexes its pattern table using the global branch history XORed with the branch's program counter.

hardware complexity Defined in this paper as the number of bits of predictor state, i.e. table entries multiplied by counter or history width.

pipeline flush The discarding of speculatively fetched instructions after a branch misprediction is detected.

saturating counter A small finite-state counter (typically 2 bits) that increments on taken branches and decrements on not-taken branches, saturating at its extremes.

tournament predictor A hybrid predictor that runs two or more predictors in parallel and uses a chooser table to select whichever has been more accurate for a given branch.

two-level adaptive predictor A predictor that indexes a pattern table using a branch's own recent history, allowing it to learn per-branch patterns rather than a single global bias.

Index

Alpha 21264, 6

control hazard, 4

gshare, 6

hardware complexity, 5

pipeline flush, 4

saturating counter, 5

tournament predictor, 6
two-level adaptive predictor, 6

References

J. L. Hennessy and D. A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann. (Chapter on instruction-level parallelism and branch prediction; edition to be confirmed by the authors.)

R. E. Kessler. The Alpha 21264 microprocessor. *IEEE Micro*, 19(2):24–36, March/April 1999. https://doi.org/10.1109/40.755465

R. E. Kessler, E. J. McLellan, and D. A. Webb. The Alpha 21264 microprocessor architecture. Compaq Computer Corporation, 1998. https://classes.engineering.wustl.edu/cse362/images/8/80/Ev6chip.pdf

S. McFarling. Combining branch predictors. Technical Report WRL Technical Note TN-36, Digital Equipment Corporation Western Research Laboratory, June 1993. https://www.ece.lsu.edu/tca/mcf.pdf

S. R. Sarangi. *Basic Computer Architecture*. https://srsarangi.github.io/archbook/archbook.pdf

J. E. Smith. A study of branch prediction strategies. In *Proceedings of the 8th Annual International Symposium on Computer Architecture (ISCA)*, pages 135–148, 1981.

T.-Y. Yeh and Y. N. Patt. Two-level adaptive training branch prediction. In *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO-24)*, pages 51–61, 1991.

T.-Y. Yeh and Y. N. Patt. A comparison of dynamic branch predictors that use two levels of branch history. In *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*, pages 257–266, 1993.